

Lekcja

Temat: Typy danych w MySQL – typy liczbowe

Materiały do lekcji wzorowane na dokumentacji MySQL:

<https://dev.mysql.com/doc/refman/8.4/en/data-types.html>

MySQL obsługuje typy danych w kilku kategoriach:

- typy liczbowe,
- typy daty i godziny,
- typy ciągów znaków (znakowe i bajtowe),
- typy przestrzenne
- typy danych JSON

Typy danych liczbowych

MySQL obsługuje wszystkie standardowe typy danych liczbowych zgodne z SQL.

Dzieli się one na dokładne typy danych liczbowych:

- TINYINT
- SMALLINT
- MEDIUMINT
- INTEGER synonim INT,
- BIGINT,
- DECIMAL synonim DEC i FIXED,
- NUMERIC,

przybliżone typy danych liczbowych:

- FLOAT,
- REAL synonim DOUBLE PRECISION,
- DOUBLE synonim DOUBLE PRECISION.

oraz typ wartości bitowej

- BIT służy do przechowywania wartości bitowych i jest obsługiwany w tabelach typu MyISAM, MEMORY, InnoDB oraz NDB.

Typ	Rozmiar (bajty)	Minimalna wartość (ze znakiem) Signed	Maksymalna wartość (ze znakiem) Signed	Minimalna wartość (bez znaku) Unsigned	Maksymalna wartość (bez znaku) Unsigned
TINYINT	1	-128	127	0	255
SMALLINT	2	-32768	32767	0	65535
MEDIUMINT	3	-8388608	8388607	0	16777215
INT	4	-2147483648	2147483647	0	4294967295
BIGINT	8	-2 ⁶³	2 ⁶³ - 1	0	2 ⁶⁴ - 1

Polecenie zwracające informację o trybie ścisłym (strict mode) w MySQL.

```
SELECT @@sql_mode;
```

albo dokładniej:

```
SHOW VARIABLES LIKE 'sql_mode';
SHOW GLOBAL VARIABLES LIKE 'sql_mode';
```

🔗 Jeśli w wyniku zobaczysz np.:

- STRICT_TRANS_TABLES
- lub STRICT_ALL_TABLES

to znaczy, że tryb ścisły jest **włączony**.

Dodatkowe przydatne polecenia

Polecenie	zwraca
SELECT @@GLOBAL.sql_mode;	Tryb globalny (dla całego serwera)
SELECT @@SESSION.sql_mode;	Tryb dla Twojego aktualnego połączenia
SELECT @@sql_mode;	Skrót do sesji

Nie ma STRICT_TRANS_TABLES ani STRICT_ALL_TABLES → tryb **nieściśły**

Różnice między STRICT_TRANS_TABLES a STRICT_ALL_TABLES

🔗 STRICT_TRANS_TABLES

- tryb ścisły działa **tylko dla tabel transakcyjnych** (np. InnoDB)
- dla innych tabel (np. MyISAM) MySQL może **zamiast błędu dać ostrzeżenie i poprawić dane**

👉 STRICT_ALL_TABLES

- tryb ścisły działa **dla wszystkich tabel**
- zawsze dostajesz **błąd**, niezależnie od typu tabeli

Zmiana na tryb ścisły dla obecnej sesji

```
SET SESSION sql_mode = CONCAT(@@sql_mode, ',STRICT_TRANS_TABLES');
```

To automatycznie **zachowa** wszystkie Twoje obecne ustawienia i tylko doda tryb ścisły.

Weryfikacja, czy się zmienił na tryb ścisły

```
SELECT @@SESSION.sql_mode;
```

Ustawienie trybu ścisłego (Strict Mode)

GLOBAL (dla całego serwera – wszystkie nowe połączenia)

- **Tylko STRICT_TRANS_TABLES** (zalecany w większości przypadków):

```
SET GLOBAL sql_mode = 'STRICT_TRANS_TABLES';
```

- **Tylko STRICT_ALL_TABLES** (najsurowszy – działa dla wszystkich tabel, w tym MyISAM):

```
SET GLOBAL sql_mode = 'STRICT_ALL_TABLES';
```

SESSION (tylko dla aktualnego połączenia)

- **STRICT_TRANS_TABLES:**

```
SET SESSION sql_mode = 'STRICT_TRANS_TABLES';
```

- **STRICT_ALL_TABLES:**

```
SET SESSION sql_mode = 'STRICT_ALL_TABLES';
```

Dodanie trybu ścisłego bez nadpisywania innych ustawień (zalecane!)

❖ GLOBAL

```
SET GLOBAL sql_mode = CONCAT(@@GLOBAL.sql_mode,  
'STRICT_TRANS_TABLES');
```

```
SET GLOBAL sql_mode = CONCAT(@@GLOBAL.sql_mode, 'STRICT_ALL_TABLES');
```

❖ **SESSION**

```
SET SESSION sql_mode = CONCAT(@@SESSION.sql_mode,  
'STRICT_TRANS_TABLES');
```

```
SET SESSION sql_mode = CONCAT(@@SESSION.sql_mode,  
'STRICT_ALL_TABLES');
```

Wyłączenie trybu ścisłego (usunięcie konkretnego trybu)

GLOBAL

```
SET GLOBAL sql_mode = REPLACE(REPLACE(@@GLOBAL.sql_mode,  
'STRICT_TRANS_TABLES', ''), ',', ',');
```

```
SET GLOBAL sql_mode = REPLACE(REPLACE(@@GLOBAL.sql_mode,  
'STRICT_ALL_TABLES', ''), ',', ',');
```

SESSION

```
SET SESSION sql_mode = REPLACE(REPLACE(@@SESSION.sql_mode,  
'STRICT_TRANS_TABLES', ''), ',', ',');
```

```
SET SESSION sql_mode = REPLACE(REPLACE(@@SESSION.sql_mode,  
'STRICT_ALL_TABLES', ''), ',', ',');
```

Przykład

```
SET SESSION sql_mode = CONCAT(@@SESSION.sql_mode, 'STRICT_ALL_TABLES');  
SET SESSION sql_mode = REPLACE(REPLACE(@@SESSION.sql_mode, 'STRICT_ALL_TABLES',  
''), ',', ',');
```

```
CREATE TABLE test_int_UNSIGNED (  
    liczba      INT UNSIGNED  
) ENGINE=InnoDB;
```

```
INSERT INTO test_int_UNSIGNED (liczba) VALUES (4294967296 );  
SELECT * from test_int_UNSIGNED;
```

```
SET SESSION sql_mode = CONCAT(@@SESSION.sql_mode, 'STRICT_ALL_TABLES');  
SET SESSION sql_mode = REPLACE(REPLACE(@@SESSION.sql_mode, 'STRICT_ALL_TABLES',  
''), ',', ',');
```

```
CREATE TABLE test_int_SIGNED (  
    liczba      INT SIGNED  
) ENGINE=InnoDB;
```

```
    liczba      INT SIGNED
) ENGINE=InnoDB;
```

```
INSERT INTO test_int_SIGNED (liczba) VALUES (-2147483999 );
SELECT * from test_int_SIGNED;
```

Tryb	Co się dzieje z danymi?	Komunikat MySQL	Efekt INSERT / UPDATE
Tryb nieściśle (sql_mode pusty lub bez STRICT_*)	Obcina wartość do najbliższej dopuszczalnej (np. maksimum lub 0) i zapisuje ją	Warning (ostrzeżenie) nr 1264 „Out of range value for column...”	Operacja kończy się sukcesem (Query OK)
Tryb ściśle (STRICT_TRANS_TABLES lub STRICT_ALL_TABLES)	Odrzuca całą operację – wartość nie jest zapisywana	ERROR 1264 (22003) „Out of range value for column...”	Operacja kończy się błędem (INSERT/UPDATE nie przechodzi)

- **Tryb ściśle (Strict SQL mode)**

Najbezpieczniejszy — MySQL zachowuje się zgodnie ze standardem SQL. Każda wartość spoza zakresu powoduje błąd i nie wstawia żadnego wiersza (lub przerywa operację). Zalecany w nowych projektach.

- **Tryb luźny (non-strict)**

MySQL nie przerywa operacji, tylko „cicho” przycina wartości do granicy zakresu. Zamiast błędu generuje tylko **warning** (ostrzeżenie). Może prowadzić do utraty danych bez Twojej wiedzy — dlatego wielu programistów go nie lubi.

ZEROFILL to **atrybut wyświetlania** (display attribute) w MySQL i MariaDB dla typów liczbowych (głównie INTEGER, TINYINT, SMALLINT, MEDIUMINT, BIGINT, a także FLOAT, DOUBLE, DECIMAL).

Działa tak:

- Gdy liczba ma **mniej cyfr** niż określona szerokość wyświetlenia, MySQL **dopełnia ją zerami z lewej strony**.
- Jest to **tylko kwestia wyświetlania** – w bazie danych wartość jest przechowywana normalnie (bez zer).

Ważne fakty:

- Gdy dodasz ZEROFILL, MySQL **automatycznie** dodaje atrybut UNSIGNED (kolumna nie może przechowywać liczb ujemnych).
- Atrybut ZEROFILL jest **przestarzały** (deprecated) w nowszych wersjach MySQL 8.0+ i MariaDB. W przyszłości może zostać całkowicie usunięty. Zaleca się go unikać.

Przykłady działania

```
CREATE TABLE test_zerofill (  
  id          INT AUTO_INCREMENT PRIMARY KEY,  
  nr1         INT(5) ZEROFILL,  
  nr2         TINYINT(3) ZEROFILL,  
  nr3         INT(8) ZEROFILL  
);
```

```
INSERT INTO test_zerofill (nr1, nr2, nr3) VALUES  
  (7, 42, 123),  
  (42, 5, 9876543),  
  (123, 100, 1);
```

```
SELECT * FROM test_zerofill;
```

id	nr1	nr2	nr3
1	00007	042	00000123
2	00042	005	09876543
3	00123	100	00000001

BIT[(M)]

Typ bitowy. M określa liczbę bitów na wartość — od **1 do 64**. Jeśli M zostanie pominięte, domyślna wartość wynosi **1**.

TINYINT[(M)] [UNSIGNED] [ZEROFILL]

Bardzo mała liczba całkowita.

- **Zakres ze znakiem (signed):** -128 do 127
- **Zakres bez znaku (unsigned):** 0 do 255

BOOL, BOOLEAN

Są to synonimy dla TINYINT(1). Wartość **zero** jest traktowana jako **false**, a każda wartość różna od zera jako **true**.

```
SELECT IF(0, 'true', 'false');    -- wynik: false
SELECT IF(1, 'true', 'false');    -- wynik: true
SELECT IF(2, 'true', 'false');    -- wynik: true
```

Wartości TRUE i FALSE są jedynie aliasami dla 1 i 0:

```
SELECT IF(0 = FALSE, 'true', 'false');    -- true, bo 0=0 (false daje 0)
SELECT IF(1 = TRUE, 'true', 'false');    -- true
SELECT IF(2 = TRUE, 'true', 'false');    -- false, bo 2=1 daje false
SELECT IF(2 = FALSE, 'true', 'false');    -- false
```

Uwaga: Wartość 2 nie jest równa ani TRUE (1), ani FALSE (0).

SMALLINT[(M)] [UNSIGNED] [ZEROFILL]

Mała liczba całkowita.

- **Zakres ze znakiem:** -32 768 do 32 767
- **Zakres bez znaku:** 0 do 65 535

MEDIUMINT[(M)] [UNSIGNED] [ZEROFILL]

Średnia liczba całkowita.

- **Zakres ze znakiem:** -8 388 608 do 8 388 607
- **Zakres bez znaku:** 0 do 16 777 215

INT[(M)] [UNSIGNED] [ZEROFILL]

Zwykła liczba całkowita (standardowa).

- **Zakres ze znakiem:** -2 147 483 648 do 2 147 483 647
- **Zakres bez znaku:** 0 do 4 294 967 295

INTEGER[(M)] [UNSIGNED] [ZEROFILL]

Synonim typu INT.

BIGINT[(M)] [UNSIGNED] [ZEROFILL]

Duża liczba całkowita.

- **Zakres ze znakiem:** -9 223 372 036 854 775 808 do 9 223 372 036 854 775 807
- **Zakres bez znaku:** 0 do 18 446 744 073 709 551 615

SERIAL jest aliasem dla: BIGINT UNSIGNED NOT NULL AUTO_INCREMENT UNIQUE

Typy DECIMAL i NUMERIC służą do przechowywania **dokładnych wartości liczbowych**. Są używane w sytuacjach, gdy kluczowe jest zachowanie **pełnej precyzji**, na przykład przy danych finansowych (pieniężnych).

W MySQL typ NUMERIC jest w pełni zaimplementowany jako DECIMAL, dlatego wszystkie informacje dotyczące DECIMAL dotyczą również NUMERIC.

Deklaracja kolumny DECIMAL

Przy tworzeniu kolumny typu DECIMAL można (i zwykle należy) określić **precyzję** oraz **skalę**.

Alias

DEC / NUMERIC / FIXED = DECIMAL

Przykład:

salary **DECIMAL**(5,2)

- **5** — to **precyzja** (precision) — całkowita liczba znaczących cyfr, które mogą być przechowywane.
- **2** — to **skala** (scale) — liczba cyfr po przecinku dziesiętnym.

Przykład:

```
CREATE TABLE test_decimal (  
    salary DECIMAL(5,2)  
) ENGINE=InnoDB;
```

```
INSERT INTO test_decimal (salary) VALUES (-999.99 );  
INSERT INTO test_decimal (salary) VALUES (999.99 );  
INSERT INTO test_decimal (salary) VALUES (9991.2);  
INSERT INTO test_decimal (salary) VALUES (-1000.50);  
INSERT INTO test_decimal (salary) VALUES (1000.00);  
INSERT INTO test_decimal (salary) VALUES (123.456 );  
INSERT INTO test_decimal (salary) VALUES (99910.999);  
SELECT * from test_decimal;
```


Wynik:

<u>salary</u>
-999.99
999.99
999.99
-999.99
999.99
123.46
999.99

Kolumna z maksymalną precyzją DECIMAL(65,30)

- **65** → maksymalna liczba **wszystkich cyfr** (przed i po przecinku)
- **30** → maksymalna liczba cyfr **po przecinku**

Przykład: Z przekroczonym zakresem precyzji DECIMAL(66,30)

```
CREATE TABLE test_max_decimal (  
    bardzo_duza_liczba DECIMAL(66,31),      -- maksymalna możliwa precyzja  
) ENGINE=InnoDB;
```

Error

Zapytanie SQL: [Copy](#)

```
CREATE TABLE test_max_decimal2 (  
    bardzo_duza_liczba DECIMAL(66,31),      -- maksymalna możliwa precyzja  
) ENGINE=InnoDB;
```

MySQL zwrócił komunikat:

```
#1426 - Too big precision 66 specified for 'bardzo_duza_liczba'. Maximum is 65
```

Przykład:

```
CREATE TABLE test_max_decimal5(  
    bardzo_duza_liczba DECIMAL(65,30)  
) ENGINE=InnoDB;
```

```
Select * from test_max_decimal5;
```

Typy FLOAT i DOUBLE służą do przechowywania **przybliżonych wartości liczbowych** (w przeciwieństwie do DECIMAL, który przechowuje wartości dokładnie).

- **4 bajtów** dla typu FLOAT (pojedyncza precyzja – single precision)
- **8 bajtów** dla typu DOUBLE (podwójna precyzja – double precision)

Standard SQL pozwala na opcjonalne podanie precyzji w bitach po słowie FLOAT:

MySQL obsługuje tę składnię, ale traktuje parametr p wyłącznie jako informację o rozmiarze przechowywania:

- **p od 0 do 23** → kolumna typu FLOAT (4 bajty, pojedyncza precyzja)
- **p od 24 do 53** → kolumna typu DOUBLE (8 bajty, podwójna precyzja)

MySQL pozwala na starszą, **niezalecaną** składnię:

```
FLOAT(M,D)
DOUBLE(M,D)
REAL(M,D)          -- REAL jest synonimem DOUBLE
DOUBLE PRECISION(M,D)
```

Gdzie:

- **M** = maksymalna liczba **wszystkich cyfr** (przed i po przecinku)
- **D** = liczba cyfr **po przecinku**

Przy zapisie MySQL wykonuje **zaokrąglanie**. Jeśli wstawisz 999.00009 do kolumny FLOAT(7,4), zostanie zapisane jako **999.0001**.

Ważna uwaga:

FLOAT(M,D) oraz DOUBLE(M,D) jest **niezgodna ze standardem i przestarzała** (deprecated). W przyszłości wsparcie dla tych wariantów może zostać całkowicie usunięte z MySQL. **Nie zaleca się** ich używania w nowych projektach.

Najlepsza praktyka (zalecana)

Dla maksymalnej przenośności kodu zaleca się używanie:

```
FLOAT          -- lub FLOAT(p)
DOUBLE         -- lub DOUBLE PRECISION
```

bez podawania precyzji (M,D) ani liczby cyfr.

Typ danych BIT służy do przechowywania **wartości bitowych** (ciągów zer i jedynek).

Składnia:

```
BIT(M)
```

- **M** określa liczbę bitów, które mogą być przechowywane w kolumnie.
- **M** może wynosić od **1 do 64**.

Do wstawiania wartości bitowych służy notacja **b'value**, gdzie value składa się wyłącznie z cyfr **0** i **1**.

Przykłady:

- b'111' → oznacza liczbę **7** (w systemie dziesiętnym)
- b'10000000' → oznacza liczbę **128**
- b'0' → 0
- b'1' → 1
- b'1010' → 10

Dopełnianie zerami (padding)

Jeśli przypiszesz do kolumny BIT(M) wartość, która ma **mniej niż M bitów**, MySQL automatycznie **dopełni ją zerami z lewej strony**.

Przykład:

Przypisanie wartości b'101' do kolumny BIT(6) jest równoważne przypisaniu b'000101'.

Czyli:

- b'101' (3 bity) → automatycznie staje się b'000101' (6 bitów)

MySQL obsługuje rozszerzenie pozwalające **opcjonalnie określić szerokość wyświetlania** (display width) dla typów całkowitoliczbowych. Szerokość podaje się w nawiasach po nazwie typu.

Przykład:

INT(4)

oznacza typ INT z szerokością wyświetlania równą 4 cyfrom.

Do czego służy szerokość wyświetlania?

Szerokość wyświetlania jest używana przez aplikacje do formatowania wyników. Jeśli wartość ma mniej cyfr niż określona szerokość, aplikacja może **dopełnić ją spacjami z lewej strony**. Ta informacja jest zwracana w metadanych wyniku zapytania — czy zostanie faktycznie użyta, zależy wyłącznie od aplikacji.

Bardzo ważne:

- Szerokość wyświetlania **nie ogranicza** zakresu wartości, jakie można zapisać w kolumnie.
- Nie zapobiega też poprawnemu wyświetlaniu większych liczb.

Przykład: Kolumna SMALLINT(3) nadal ma pełny zakres od -32768 do 32767. Wartości spoza 3 cyfr (np. 12345) będą wyświetlane w całości, z większą liczbą cyfr.

```
CREATE TABLE test_SMALLINT(
    mala_liczba SMALLINT(3)
) ENGINE=InnoDB;
```

```
INSERT INTO test_SMALLINT ( mala_liczba ) VALUES
    ( 12345),
    ( 12.1 );
Select * from test_SMALLINT;
```

<u>mala_liczba</u>	
12345	
12	

Atrybut ZEROFILL

Gdy użyjemy atrybutu ZEROFILL razem z szerokością wyświetlania, zamiast spacji wartości są dopełniane **zerami** z lewej strony.

Przykład:

```
INT(4) ZEROFILL
```

Wartość 5 zostanie zwrócona jako **0005**.

Uwaga: Atrybut ZEROFILL jest ignorowany, gdy kolumna jest używana w wyrażeniach ub zapytaniach UNION.

Przepelnienie (Overflow) podczas obliczeń

Przykład z największą wartością BIGINT (ze znakiem):

```
SELECT 9223372036854775807 + 1;    -- błąd
```

Rozwiązania:

- Rzutowanie na UNSIGNED:

```
SELECT CAST(9223372036854775807 AS UNSIGNED) + 1;  -- zwraca  
9223372036854775808
```

- Użycie arytmetyki dokładnej (DECIMAL):

```
SELECT 9223372036854775807.0 + 1;
```